L.D. College of Engineering, Ahmedabad

Department of Artificial Intelligence and Machine Learning

Machine Learning (BE05000211)

Project Report

REPORT ON LEADIQ

Explainable AI Lead Scoring & Prioritization System using Machine Learning, Text Embeddings and SHAP

Submitted By:

Sr. No.	Name of Student	Enrollment No.
1	Darshil Savaliya	240280152068
2	Harshil Patel	240280152052
3	Aranav Mandani	240280152037

Branch: Artificial Intelligence and Machine Learning

Semester: 5th

Academic Year: 2026-27

Subject: Machine Learning (BE05000211)

Faculty Name & Sign: _____

Date: _____

CONTENTS

Sr. No.	Content	Page No.
1	Introduction	3
2	Objectives	5
3	Technologies and Tools	6
4	System Requirements and Setup	9
5	System Design	11
6	Implementation and Working of the System	16
7	Results and Output Screens	22
8	Advantages, Limitations and Future Scope	30
9	Conclusion	32
10	References	33

1. Introduction

1.1 Background

Every sales and outreach team works from a list of possible customers, or *leads*. A lead is a business or a person who might eventually buy a product or respond to an offer. In practice, far more leads are collected than can ever be contacted: a spreadsheet may hold thousands of rows, yet only a small fraction will ever convert into a real conversation or sale. The central question for any team is therefore not *who is on the list*, but *who should be contacted first*. Answering that question well saves time, raises conversion rates and keeps effort focused where it matters.

Traditionally this prioritization is done by hand, using simple rules such as “call the highest-rated businesses first”. Such rules ignore most of the information in the data and cannot weigh many signals at once. Modern machine learning can do better: given historical records whose outcome is already known, a model can learn which combinations of attributes are associated with conversion and then score new leads by their estimated probability of converting. LeadIQ, the system described in this report, turns an uploaded spreadsheet into a ranked, explainable contact list, with the tagline “Your next best lead — and the why behind it.”

1.2 Problem Statement

Scoring leads with machine learning is useful only if the result can be **trusted**. Three problems make naive solutions unreliable. First, real lead data is **messy and mixed**: it contains numbers, categories, free text and missing values, and it is almost always **imbalanced** — far fewer leads convert than do not. Second, a model that is only measured on the data it was trained on will look far better than it really is. Third, a bare probability answers *who* but never *why*, so a user cannot judge whether a score is sensible or act on it with confidence. LeadIQ addresses all three: it cleans and encodes arbitrary tabular data, it measures every model on an untouched test set with honest uncertainty, and it explains each prediction with real SHAP values.

1.3 Motivation and Relevance to Machine Learning

The project is a complete, end-to-end example of the **Knowledge Discovery in Databases (KDD)** process, the classic framework that turns raw data into usable knowledge through selection, pre-processing, transformation, data mining and interpretation. LeadIQ maps its nine guided steps directly onto these KDD stages. The engineering work lies around the models: preparing and cleaning the input, building features (including sentence-transformer text embeddings), comparing four classifiers against a no-skill baseline with cross-validation, calibrating probabilities, choosing an operating threshold, and explaining the winner — so that model output can be trusted and acted upon in practice.

1.4 Scope of the Project

- Upload of any tabular dataset (CSV, TSV, TXT, XLSX or XLS, up to 10 MB) with automatic parsing, sheet selection and column-role detection — the system is not tied to a fixed schema.
- Interactive selection of the target column and the positive outcome, with automatic leakage checks.
- Data cleaning with a transparent quality report, and exploratory analysis of outcome patterns.
- Automatic feature engineering: numeric scaling, categorical target-encoding, presence/validity flags for contact fields, and NLP embeddings for free-text columns.
- Training and fair comparison of Logistic Regression, Decision Tree, Random Forest and XGBoost against a dummy baseline, with training-only cross-validation and a reserved test split.
- A ten-tab evaluation workspace, probability calibration, an operating threshold, lead scoring into priority tiers, per-lead SHAP explanations, and CSV / Excel / PDF export.

2. Objectives

2.1 Aim

The aim of the project is to build a web-based system that converts an uploaded, outcome-labelled dataset into a prioritized and explainable contact list, so that a team can decide whom to contact next with measured confidence, while keeping the user in control of every decision.

2.2 Specific Objectives

1. To accept arbitrary tabular files, parse them robustly, and let the user confirm the target column and the positive class.
2. To clean the data — handling duplicates, missing values and inconsistent types — and to report exactly what was changed.
3. To build a numeric feature matrix automatically, including target-encoded categories and sentence-transformer embeddings of free-text columns.
4. To reserve an untouched test set and use training-only cross-validation so that reported performance is honest and not optimistic.
5. To train and compare four classifiers against a no-skill baseline and select a winner by cross-validated average precision (PR-AUC).
6. To calibrate the winner’s probabilities and choose an operating threshold suited to the class imbalance.
7. To score leads into percentile scores and priority tiers, and to rank them in a deterministic contact order.
8. To explain each individual prediction with real SHAP values and to disclose the model’s limitations honestly.
9. To export the ranked leads and a summary report as CSV, Excel and PDF.

3. Technologies and Tools

3.1 Supervised Classification and the KDD Process

LeadIQ treats lead scoring as a **binary classification** problem: each labelled record belongs to one of two classes (the positive outcome, here “converted”, or the negative one). A model learns a function from the input features to a probability of the positive class. The whole application follows the KDD process — *selection, pre-processing, transformation, data mining* and *interpretation / knowledge use* — which gives the nine-step workflow its academic backbone and keeps each stage separate and inspectable.

3.2 Feature Engineering and Text Embeddings

Numeric columns are scaled; categorical columns are converted to numbers with scikit-learn’s **target (mean) encoding**, which replaces each category with a smoothed estimate of the outcome rate for that category and is well suited to high-cardinality fields. Contact columns such as phone and e-mail are turned into *presence* and *validity* flags. Free-text columns are encoded with a pre-trained **sentence-transformer** model, *all-MiniLM-L6-v2*, which maps each text to a 384-dimensional embedding vector that captures its meaning; these vectors become additional numeric features. In this project 54 engineered features were produced from the raw columns.

3.3 Models Compared

Four classifiers are trained and compared: **Logistic Regression** (a linear, highly interpretable model), a **Decision Tree**, a **Random Forest** (an ensemble of trees), and **XGBoost** (gradient-boosted trees). A **Dummy baseline** that always predicts the majority class is included as a no-skill reference, so that real skill can be distinguished from the effect of class imbalance.

3.4 Honest Evaluation, Calibration and SHAP

Because conversion is rare, **accuracy alone is misleading** — a model that predicts “no” for everyone can still look accurate. LeadIQ therefore relies on **ROC-AUC** and, especially, **PR-AUC (average precision)**, together with precision, recall, F1 and *precision in the top 10%*. Models are selected by training-only cross-validation and reported on a reserved test set with 95% bootstrap confidence intervals. Predicted probabilities are **calibrated** (isotonic regression) so that a score of, say, 0.3 really means about a 30% chance. Finally, **SHAP** (SHapley Additive exPlanations) assigns to every feature its contribution to a specific prediction, providing the “why” behind each score.

3.5 Web Application Stack

The frontend is a single-page **React** application styled with Tailwind CSS, using Recharts for interactive charts and Framer Motion for animation. The backend is a **FastAPI** service that runs the machine-learning pipeline with scikit-learn, XGBoost, SHAP and sentence-transformers, and stores project metadata in **MongoDB**. A light theme and a nine-step guided journey make the workflow easy to follow.

3.6 Summary of the Technology Stack

Table 3.1 lists the main components with the versions used in the project.

Component	Technology	Purpose
Frontend framework	React 18, React Router	Single-page nine-step wizard UI
Styling & charts	Tailwind CSS, Recharts, Framer Motion	Layout, interactive charts, animation
Web framework	FastAPI (Python)	REST API for the ML pipeline
ML & data	scikit-learn 1.6.1, pandas 3.0.6, NumPy 2.2.6	Pipelines, encoders, models, data handling
Gradient boosting	xgboost-cpu 3.0.4	XGBoost classifier
Explainability	shap 0.48.0	SHAP feature contributions
Text embeddings	sentence-transformers 3.4.1, transformers 4.49.0	all-MiniLM-L6-v2 embeddings of free text
Persistence	MongoDB, joblib 1.4.2	Project metadata and trained-model artefacts
Export	openpyxl 3.1.5, reportlab	Excel workbooks and PDF reports

Table 3.1: Technology stack of LeadIQ

4. System Requirements and Setup

4.1 Hardware Requirements

All computation runs locally on CPU; no GPU is required. The following are recommended:

- A computer or server able to run Python 3.11+, for example a dual- or quad-core processor with 4–8 GB of RAM (the text-embedding model is the most memory-intensive stage).
- Free disk space for the application, the uploaded dataset and the cached trained models.
- A current web browser (Chrome, Edge, Firefox or Safari) to use the interface.

4.2 Software Requirements

Item	Requirement
Operating system	Windows, Linux or macOS
Language runtime	Python 3.11 or newer, with pip
Node.js / Yarn	To build and run the React frontend
Database	MongoDB (local or hosted)
Web browser	A current version of Chrome, Edge, Firefox or Safari

Table 4.1: Software requirements

4.3 Configuration Parameters

All settings are read from environment variables (a `.env` file). Table 4.2 lists the key variables.

Variable	Location	Meaning
MONGO_URL	backend/.env	MongoDB connection string
DB_NAME	backend/.env	Database name used by the backend
REACT_APP_BACKEND_URL	frontend/.env	Base URL the frontend calls for the API

Table 4.2: Configuration parameters

4.4 Installation and Execution

The application is installed and started with the following steps:

```
# Backend
cd backend
pip install -r requirements.txt
uvicorn server:app --host 0.0.0.0 --port 8001

# Frontend
cd frontend
yarn install
yarn start
```

The backend exposes its REST API under the `/api` prefix; the React frontend talks to it through the `REACT_APP_BACKEND_URL` variable.

5. System Design

5.1 Overall Workflow

Figure 5.1 shows the journey of a dataset from upload to a ranked, explainable contact list. The nine steps map directly onto the five stages of the KDD process.

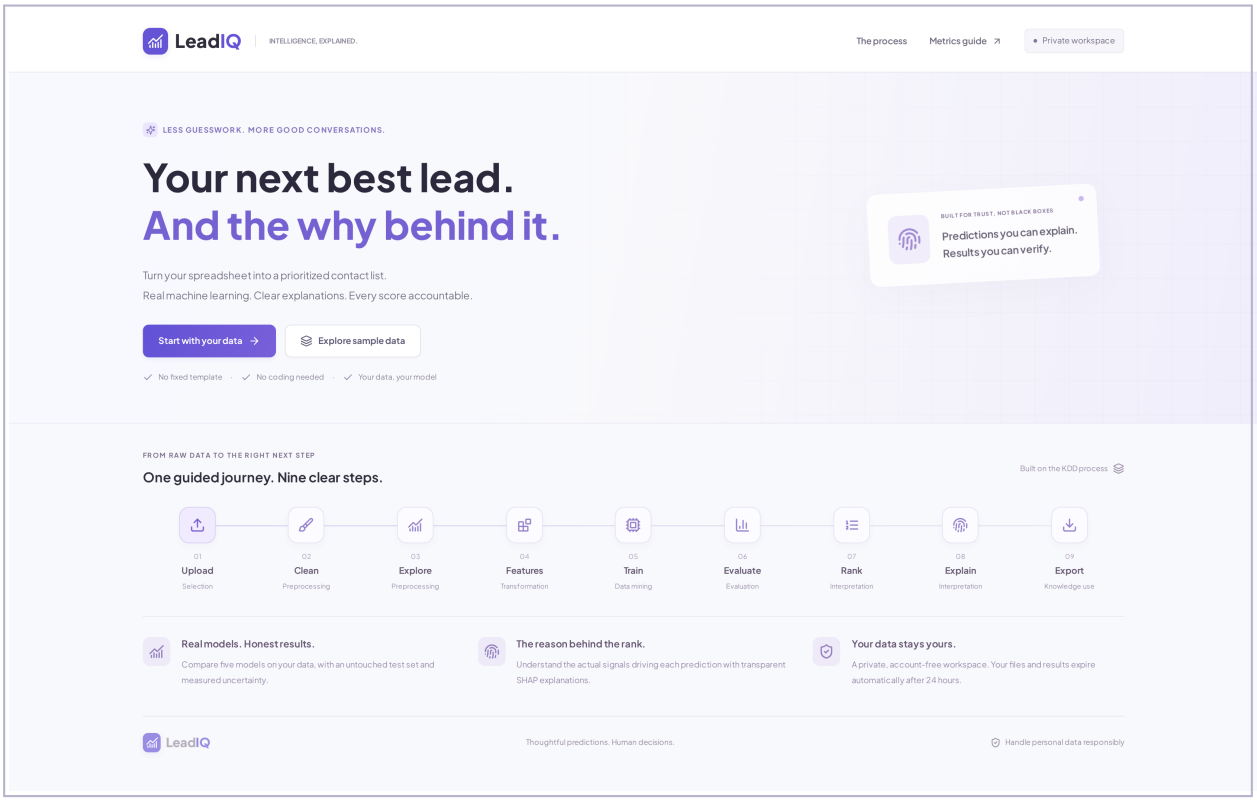


Figure 5.1: The nine-step LeadIQ journey mapped onto the KDD process (landing page).

Table 5.1 maps each step to its KDD stage and to the backend module that performs the work.

Step	Stage (KDD)	What happens	Module
1. Upload & select	Selection	Parse file; choose target and positive class	dataio.py
2. Clean	Pre-processing	Duplicates, missing values, type fixes; quality report	cleaning.py
3. Explore	Pre-processing	Outcome rates and patterns across columns	inspections.py
4. Features	Transformation	Scaling, target encoding, flags, text embeddings	features.py
5. Train	Data mining	Cross-validation, model comparison, calibration	training.py
6. Evaluate	Evaluation	Ten-tab metrics, curves, threshold explorer	evaluation.py
7. Rank	Interpretation	Calibrated probabilities, percentile scores, tiers	scoring.py
8. Explain	Interpretation	Per-lead SHAP contributions	explanations.py
9. Export	Knowledge use	CSV, Excel and PDF downloads	exports.py

5.2 System Architecture

LeadIQ follows a layered architecture. The React browser application talks to a FastAPI backend over HTTP and JSON. The route handlers in `server.py` are thin: they validate requests, enforce a per-project lock so that a running training job cannot be disturbed, and call the service modules where the real work happens. The service layer uses scikit-learn, XGBoost, SHAP and sentence-transformers, and persists project state in MongoDB and trained artefacts on disk through joblib. Each project is identified by an id and a secret token and expires automatically after 24 hours.

5.3 Project Structure

Path	Contents
backend/server.py	FastAPI application and all REST endpoints
backend/dataio.py	File parsing, sheet handling, target and leakage detection
backend/cleaning.py	Data cleaning and the quality report
backend/inspections.py	Exploratory analysis (outcome rates, profiles)
backend/features.py	Feature engineering, encoders and text embeddings
backend/training.py	Cross-validation, model comparison and calibration
backend/evaluation.py	Metrics, curves and threshold recalculation
backend/explanations.py	SHAP and permutation importance
backend/scoring.py	Probability scoring, percentile scores and tiers
backend/exports.py	CSV, Excel and PDF exports
frontend/src/components/	Landing page and the nine wizard steps

Table 5.2: Folder and file structure of the repository

5.4 Data Model

Project state is stored in MongoDB. Two main collections are used, summarised in Table 5.3.

Collection	Key fields
projects	id, token, created_at, expires_at, name, step, completed, target, positive, settings
metrics	project_id, best_model, split sizes, prevalence, roc_auc, pr_auc, calibration, threshold

Table 5.3: Main stored collections

5.5 REST API Design

All endpoints are served under the `/api` prefix. Table 5.4 lists the most important ones.

Method	Endpoint	Description
POST	/api/projects	Create a new project and return its token
GET	/api/projects/{pid}	Return the project state and current step
POST	/api/projects/{pid}/upload	Upload a dataset file
POST	/api/projects/{pid}/configure	Set the target column and positive class
POST	/api/projects/{pid}/clean	Run data cleaning

Method	Endpoint	Description
GET	/api/projects/{pid}/eda	Return exploratory analysis
POST	/api/projects/{pid}/features	Build the feature matrix
POST	/api/projects/{pid}/train	Train and compare the models
GET	/api/projects/{pid}/metrics	Return evaluation metrics and curves
POST	/api/projects/{pid}/threshold	Recompute metrics at a new threshold
GET	/api/projects/{pid}/importance	Return SHAP and permutation importance
POST	/api/projects/{pid}/score	Score and rank the leads
GET	/api/projects/{pid}/explain/{lead_id}	Explain a single lead with SHAP
GET	/api/projects/{pid}/export	Download ranked leads / report

5.6 User Interface Design

The interface is a single guided journey of nine steps shown in a stepper, with a light theme, a metrics guide, and clear loading, recalculating, unavailable and invalidated states. Changing a model-affecting choice (such as the target) invalidates the downstream steps so that stale results can never be mistaken for current ones. Table 5.5 lists the steps.

Step	Purpose	Main elements
Upload & select	Bring in the data	File drop zone, sample data, target and positive-class pickers
Clean	Fix data problems	Quality report, cleaning actions
Explore	Understand patterns	Outcome-rate charts by category, city, state
Features	Build the matrix	Feature groups, embedding status
Train	Compare models	Quick / thorough choice, training status and logs
Evaluate	Judge performance	Ten tabs, model selector, threshold explorer
Rank	Prioritize leads	Score cards, distribution chart, ranked table
Explain	See the why	Lead picker, score, SHAP contribution chart
Export	Keep the evidence	CSV, Excel and PDF downloads

Table 5.5: Steps of the user interface

6. Implementation and Working of the System

This chapter follows the order in which a dataset travels through the LeadIQ pipeline. For this report the system was run end to end on a realistic synthetic dataset of 6,000 business leads (`synthetic-business-leads.csv`) whose target column is **Converted** with positive class **yes**.

6.1 Upload, Parsing and Selection

A dataset is uploaded with `POST /api/projects/{pid}/upload`. The `dataio` module parses CSV, TSV, TXT and Excel files, detects the delimiter and sheet, infers each column's role and type, and previews the data. The user then confirms the target column and the positive outcome. The system automatically checks for **leakage** — columns that would not be known at prediction time — and excludes them. In this run 6,000 labelled rows were loaded with no leakage detected.

6.2 Data Cleaning

The `cleaning` module removes duplicate rows, reconciles inconsistent types, and handles missing values, producing a transparent quality report that records exactly what was changed. Rows with a missing label are excluded from training. Duplicate removal is done in a way that preserves the original train/test membership, so the reserved test set stays untouched.

6.3 Exploratory Analysis

Step 3 summarises the cleaned data: the overall positive-outcome rate and the outcome rate broken down by each categorical column. For this dataset 9.6% of the cleaned, labelled records have the positive outcome — a strong class imbalance of about 9.4 : 1 — which is why ranking metrics, not raw accuracy, drive the rest of the pipeline.

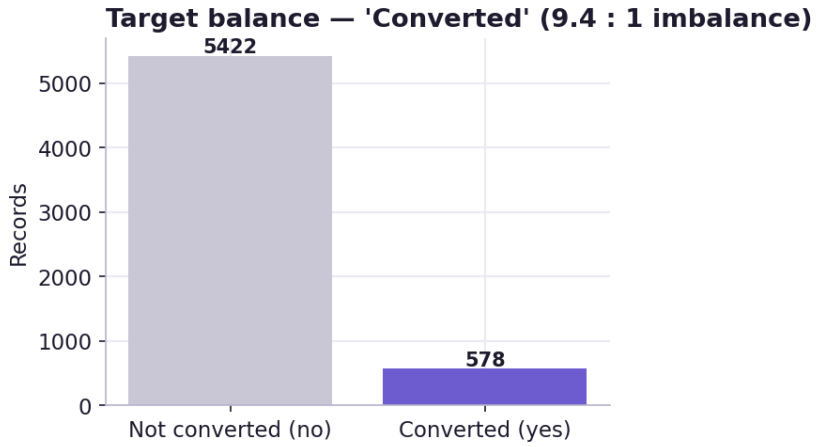


Figure 6.1: The target is strongly imbalanced — only 578 of 6,000 records converted.

6.4 Feature Engineering

The `features` module turns the cleaned table into a purely numeric matrix inside a scikit-learn pipeline, so that exactly the same transformation is applied to training, test and new data:

- **Numeric** columns (Rating, Review Count, Business Age, …) are imputed and scaled.
- **Categorical** columns (Category, City, State, Lead Source) are converted with smoothed **target encoding**.
- **Contact** columns (Website, Phone, Email, Social Media) become *presence* and *validity* flags — for example whether a phone number is valid.
- The free-text **Description** column is embedded with the pre-trained *all-MiniLM-L6-v2* sentence-transformer into a 384-dimensional vector.

Together these produced **54 engineered features**. The embedding stage, which runs on CPU, took about 69 seconds for this dataset. A stratified 80/20 split reserves an untouched test set before any model sees the data.

6.5 Model Training and Selection

The `training` module runs **training-only cross-validation** to compare the four classifiers against the dummy baseline. Each model's hyper-parameters are tuned over a small search (8 candidates in the "quick" mode used here) using k-fold cross-validation *within the training set only*. The winner is chosen by **cross-validated average precision (PR-AUC)** — never by test-set performance — after which it is refit and its probabilities are **calibrated with isotonic regression**. For this run **Logistic Regression** (C = 0.01) was selected, and the whole process took about 84 seconds.

6.6 Evaluation and the Threshold Explorer

The reserved test set is scored once, at the very end, to give an honest estimate of real-world performance. The `evaluation` module computes accuracy, precision, recall, F1, ROC-AUC, PR-AUC and precision in the top 10%, together with ROC, precision–recall, lift, cumulative-gain, learning and calibration curves, and 95% bootstrap confidence intervals. An **operating threshold** (0.114 here, chosen out-of-fold to suit the imbalance) converts probabilities into yes/no decisions. A **threshold explorer** lets the user move the threshold and immediately see the confusion matrix and threshold-dependent metrics recompute from the saved probabilities — without changing the probabilities, the ranking or the saved operating point.

6.7 Scoring and Ranking

The `scoring` module applies the calibrated winner to produce, for each lead, a calibrated probability and a **percentile score from 0 to 100** (a rank relative to the training distribution, explicitly not a probability). Scores are grouped into four priority tiers — Very High, High, Medium and Low — and the leads are placed in a deterministic contact order. For the 1,167 held-out records, 136 fell into Very High, 257 into High, 325 into Medium and 449 into Low.

6.8 Explanation with SHAP

For any chosen lead the `explanations` module computes **SHAP** values, showing how much each feature pushed that particular prediction up or down. The interface states clearly that SHAP explains the underlying *uncalibrated* estimator, not the calibrated probability or the percentile score, and that contributions are associations, not causation. Global importance is also provided (SHAP and permutation), with sampling limitations disclosed.

6.9 Export

Finally the `exports` module writes the ranked leads to CSV, a formatted Excel workbook (with a frozen header and safe cells) and a PDF summary containing the key metrics, the model comparison and the top-ranked records, so that the evidence travels with the result.

7. Results and Output Screens

This chapter presents the real output of the deployed LeadIQ application, in the order a user experiences it, together with the charts produced from the actual run described in Chapter 6.

7.1 Upload & Select

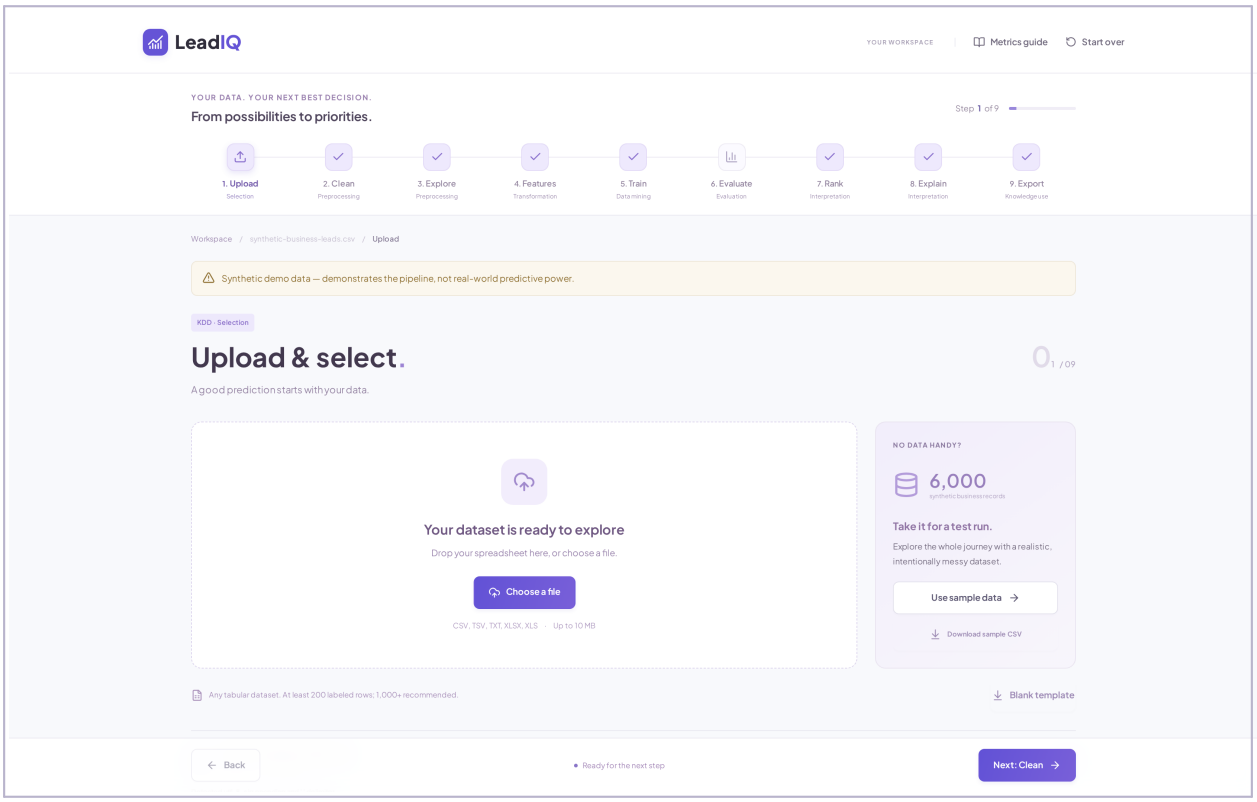


Figure 7.1: Step 1 — uploading a dataset and selecting the target and positive class.

7.2 Explore the Data

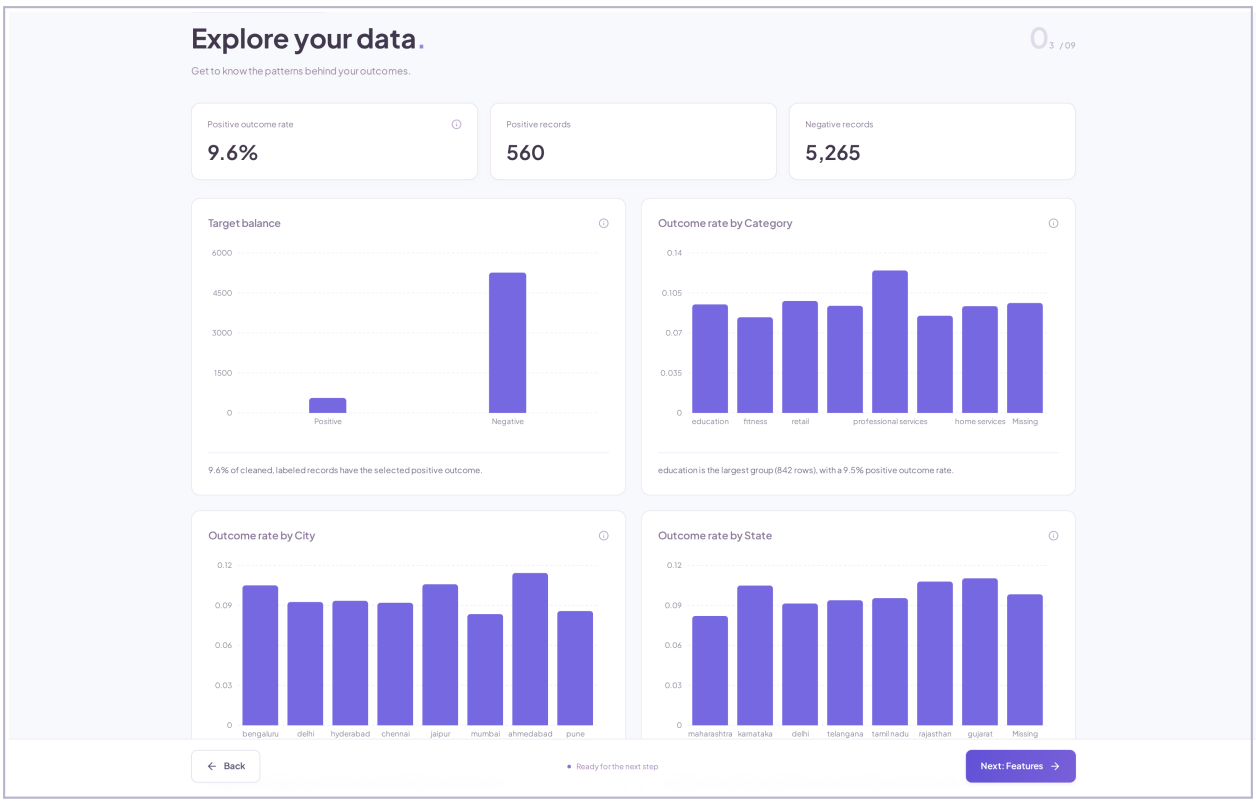


Figure 7.2: Step 3 — exploratory analysis of outcome rates across categories, cities and states.

7.3 Train the Models

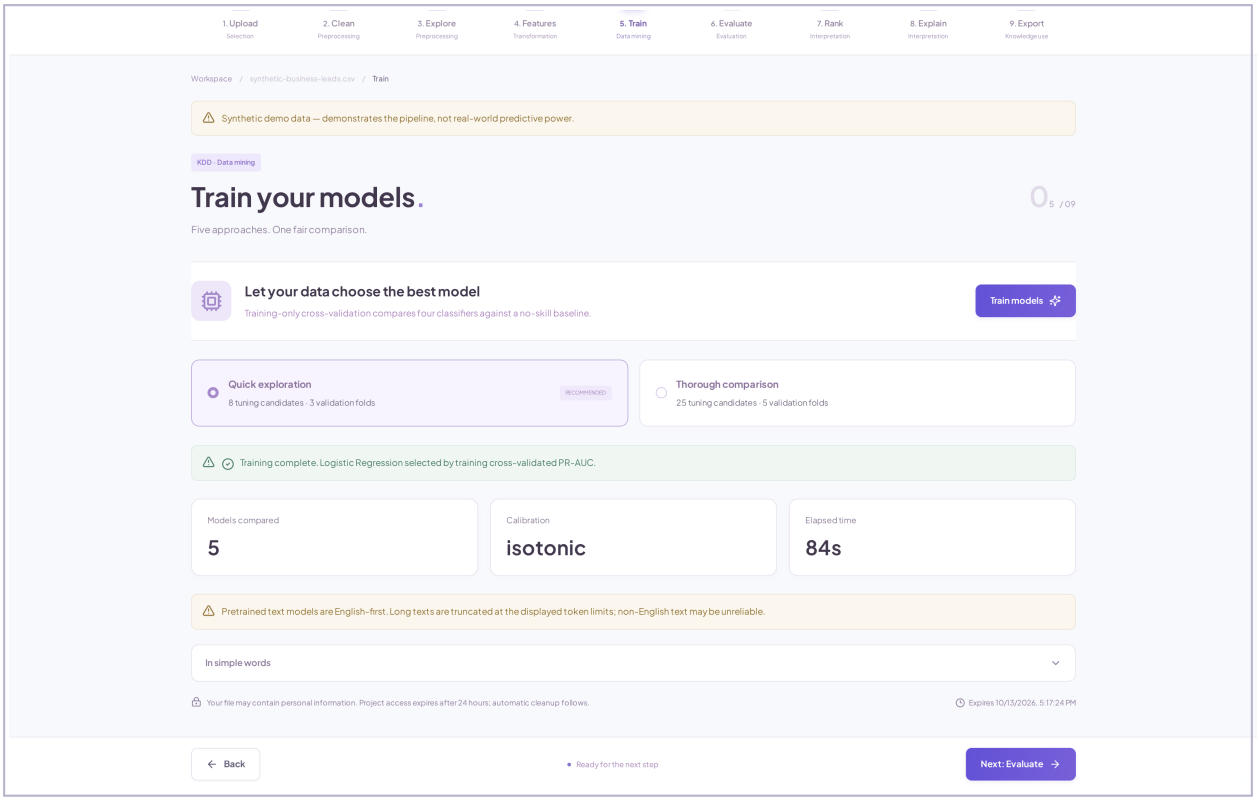


Figure 7.3: Step 5 — training complete; Logistic Regression selected by cross-validated PR-AUC in 84 s.

7.4 Model Comparison

All five candidates were measured on the same untouched test set. Table 7.1 and Figure 7.4 report the results; the Dummy baseline shows why accuracy alone is misleading (it reaches 0.901 accuracy with zero recall).

Model	Acc.	Prec.	Recall	F1	ROC-AUC	PR-AUC	P@10%	CV PR-AUC
Logistic Regression (winner)	0.743	0.208	0.574	0.306	0.711	0.271	0.291	0.187
Decision Tree	0.683	0.169	0.565	0.260	0.647	0.156	0.214	0.148
Random Forest	0.862	0.298	0.296	0.297	0.699	0.232	0.291	0.154
XGBoost	0.681	0.171	0.583	0.265	0.717	0.271	0.282	0.176
Dummy baseline	0.901	0.000	0.000	0.000	0.500	0.099	0.128	0.096

Table 7.1: Test-set results for all models (winner selected by CV PR-AUC).

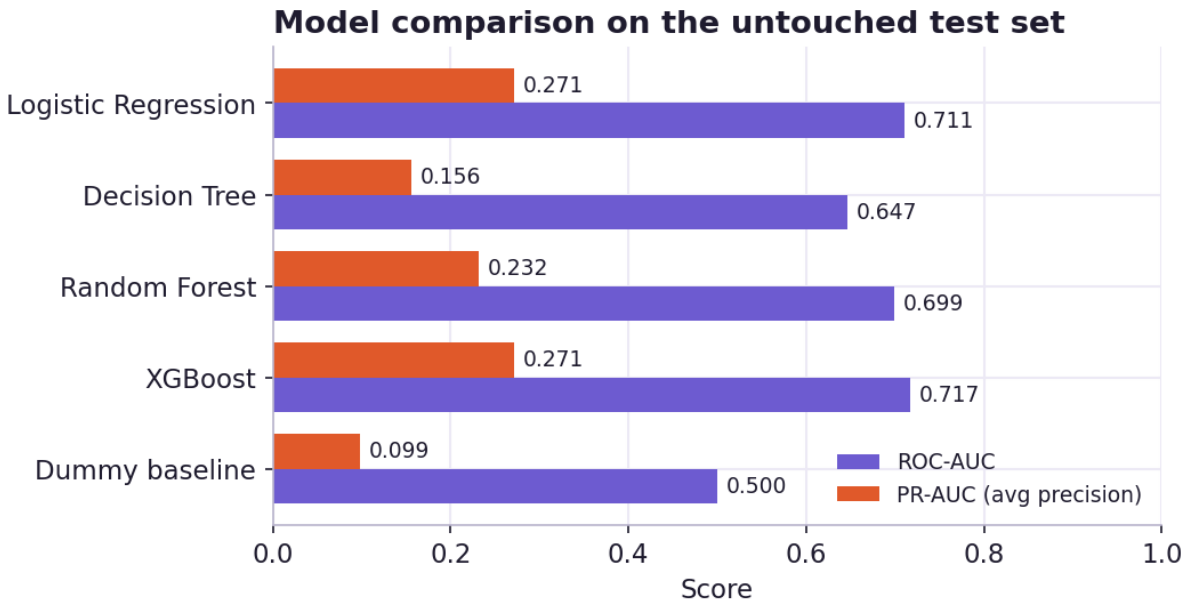


Figure 7.4: ROC-AUC and PR-AUC for each model on the untouched test set.

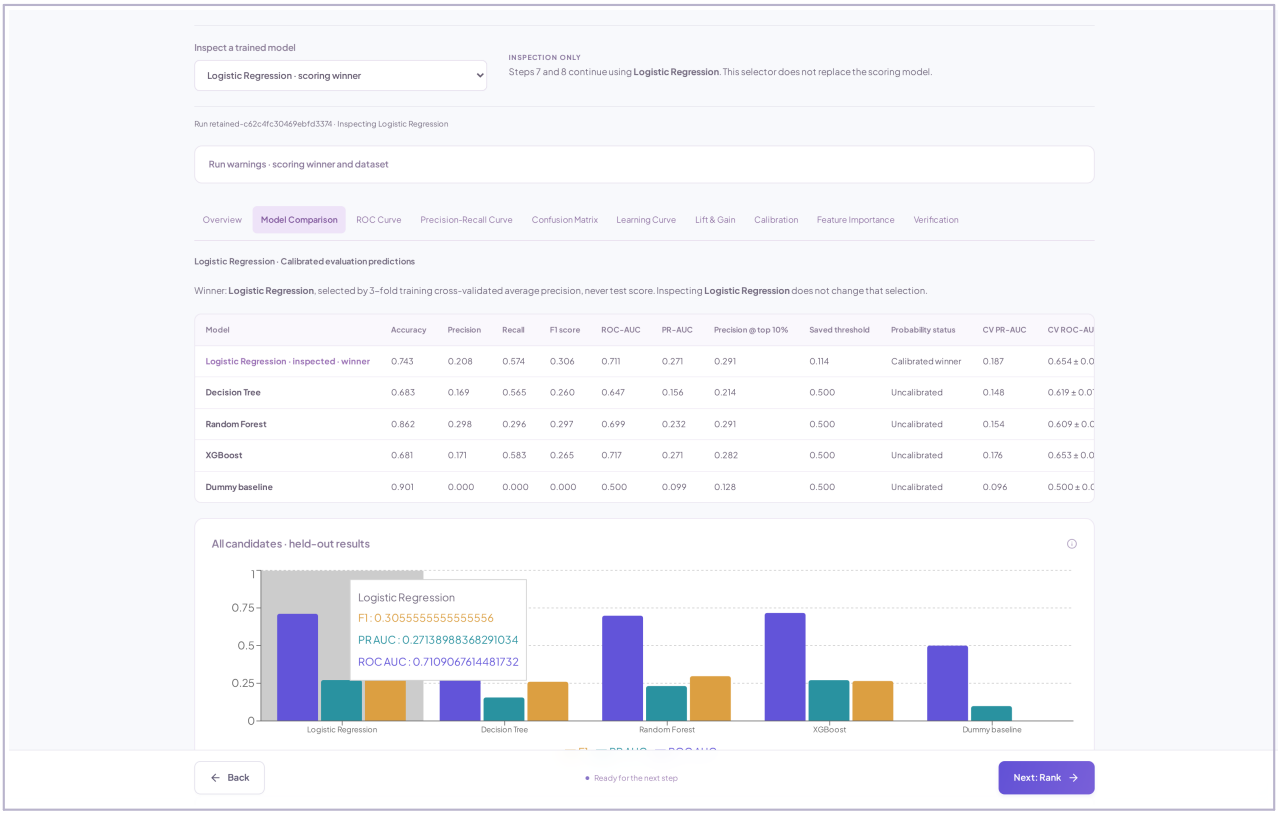


Figure 7.5: Step 6 — the Model Comparison tab in the application.

7.5 Evaluation Metrics of the Selected Model

Table 7.2 summarises the winning model's test-set metrics with 95% bootstrap confidence intervals (1,000 resamples), and Figure 7.6 shows the Overview tab.

Metric	Value	95% confidence interval
Accuracy	0.743	–
Precision	0.208	–
Recall	0.574	–
F1 score	0.306	0.252 – 0.363
ROC-AUC	0.711	0.660 – 0.765
PR-AUC (avg precision)	0.271	0.208 – 0.354
Precision @ top 10%	0.291	2.95× lift over random
Operating threshold	0.114	chosen out-of-fold

Table 7.2: Test-set metrics of the selected Logistic Regression model.

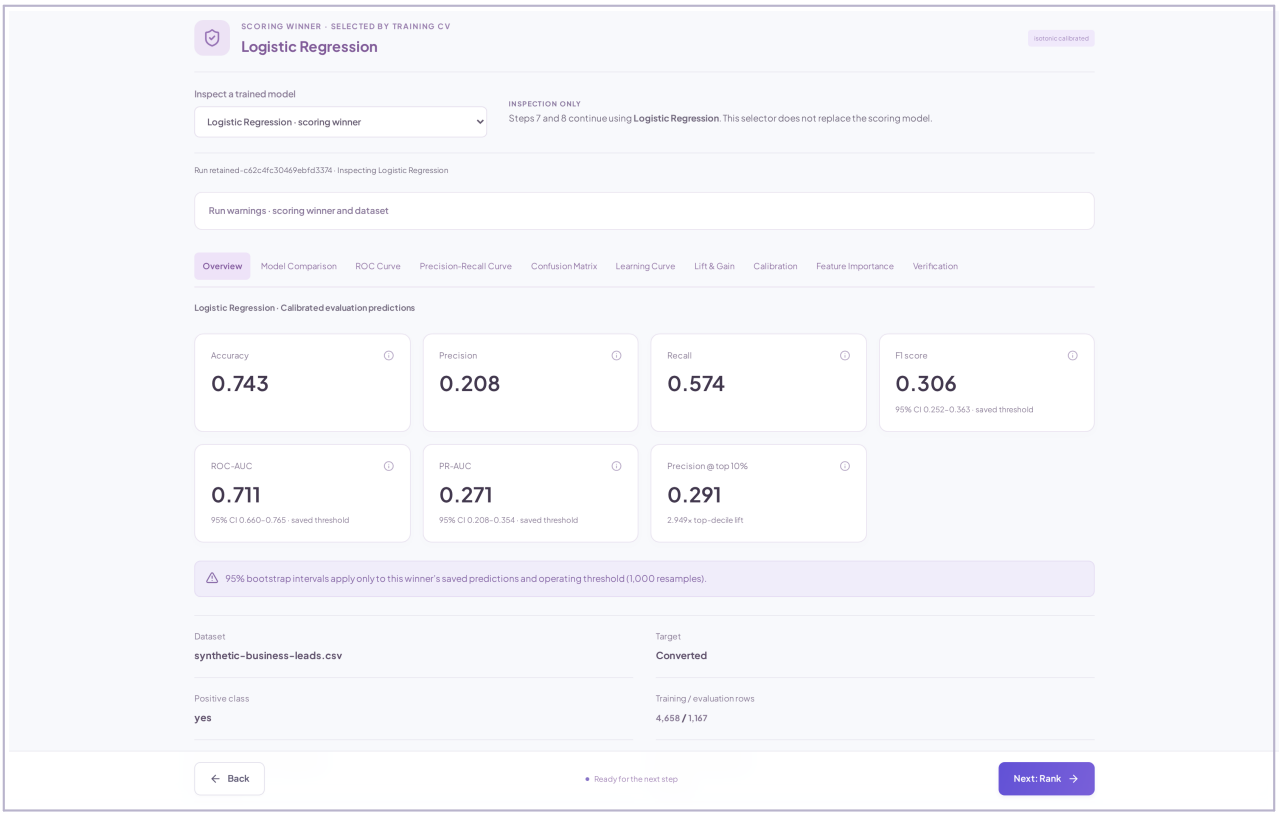


Figure 7.6: Step 6 — the Overview tab with the seven headline metrics and dataset context.

7.6 Curves and Diagnostics

The figures below are drawn from the actual saved predictions of the selected model.

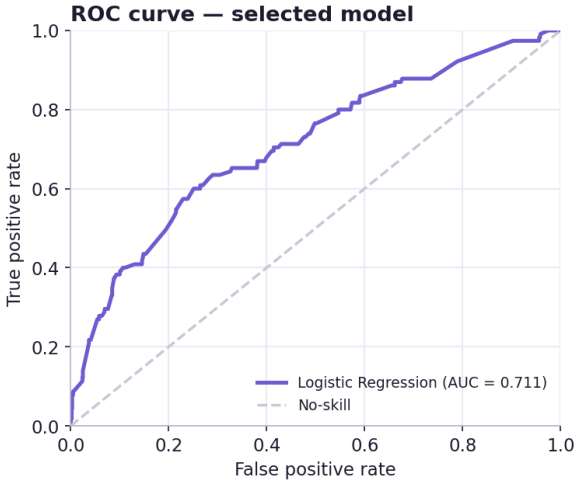


Figure 7.7: ROC curve (AUC = 0.711).

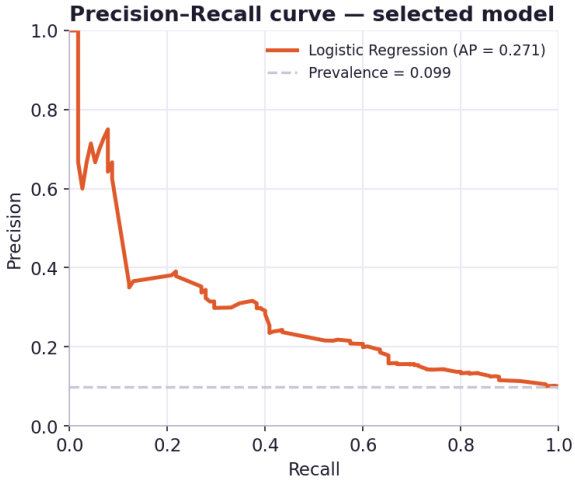


Figure 7.8: Precision–Recall curve (AP = 0.271).

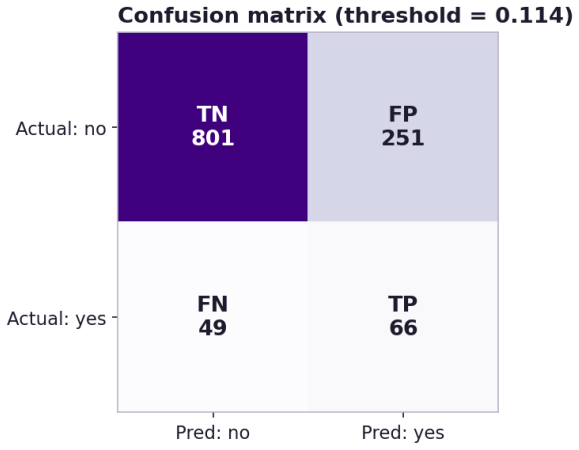


Figure 7.9: Confusion matrix at the operating threshold.

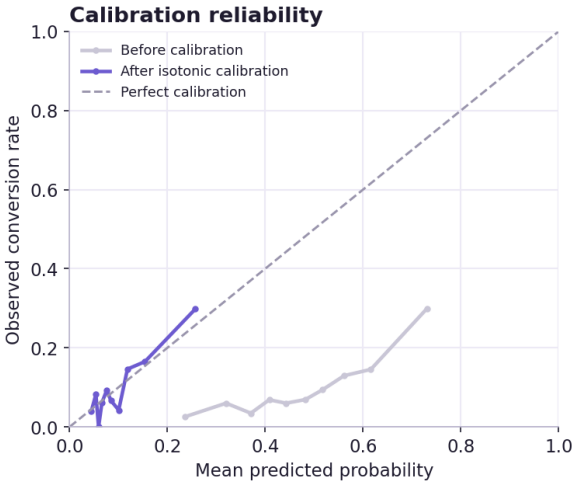


Figure 7.10: Calibration reliability (before / after isotonic).

The confusion matrix at the 0.114 threshold yields 66 true positives and 801 true negatives, against 251 false positives and 49 false negatives — a recall-leaning operating point that deliberately surfaces more candidates, as is sensible for prioritizing outreach.

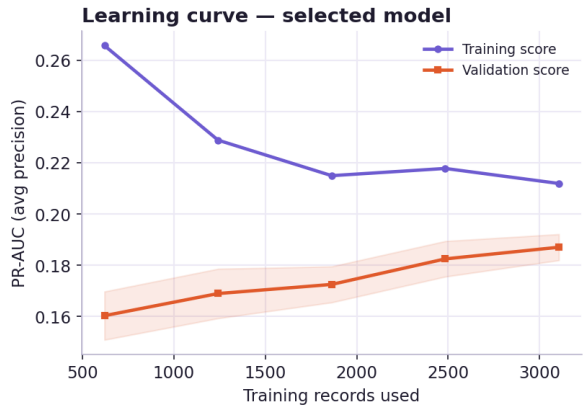


Figure 7.11: Learning curve of the selected model.

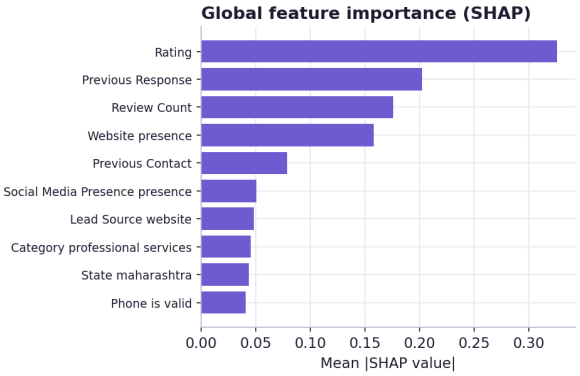


Figure 7.12: Global feature importance (mean |SHAP|).

7.7 Lift and Cumulative Gain

Lift and cumulative-gain curves show the business value of the ranking: by contacting only the top-scored leads, far more conversions are captured than by contacting at random.

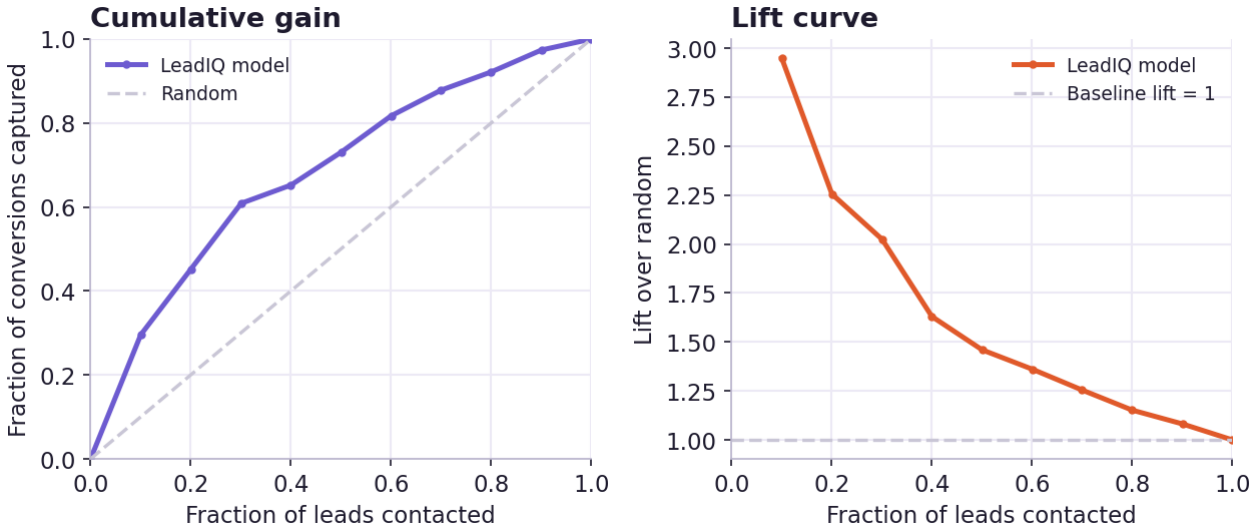


Figure 7.13: Lift (left) and cumulative gain (right) for the LeadIQ ranking.

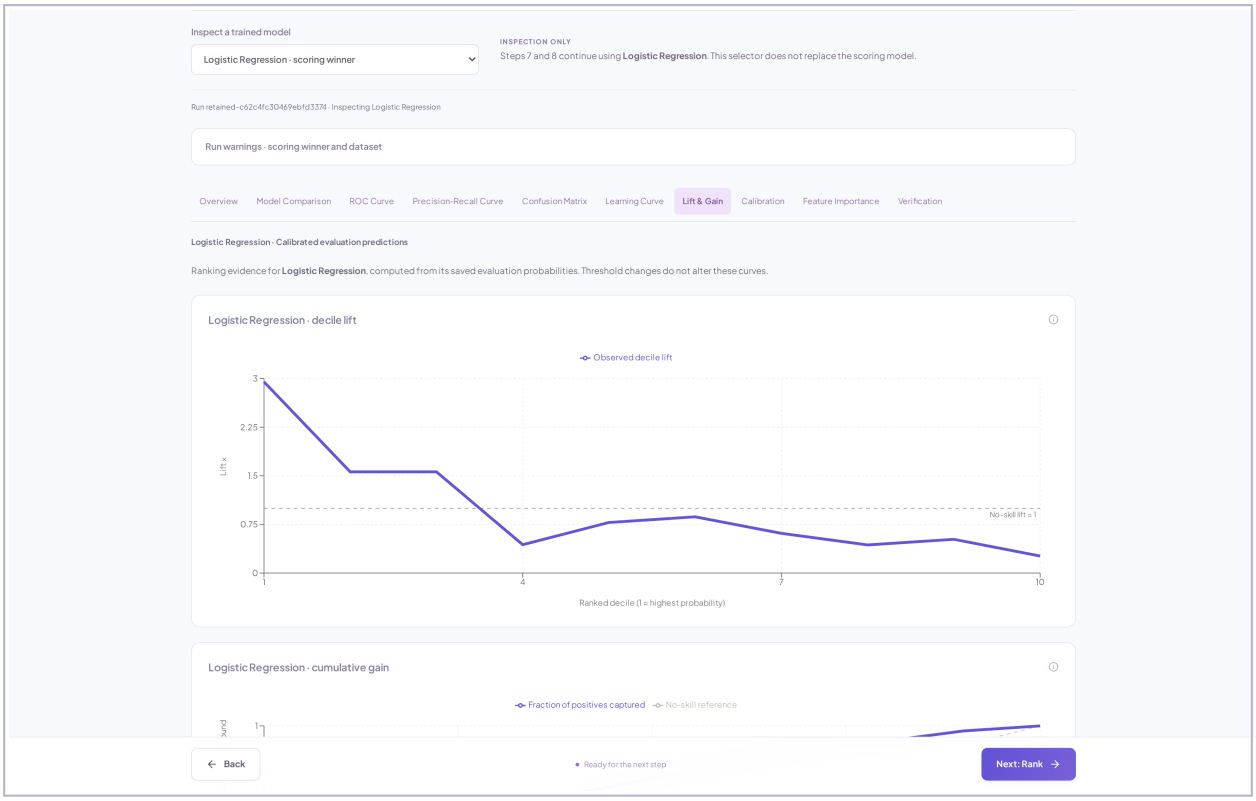


Figure 7.14: Step 6 — the Lift & Gain tab in the application.

7.8 Score & Rank Leads

The selected model scored the 1,167 held-out leads into percentile scores and four priority tiers. Figure 7.15 shows the distribution and Figure 7.16 the ranked list.

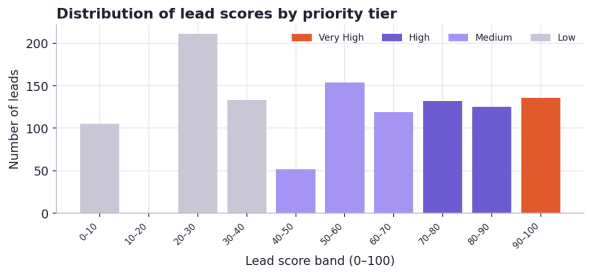


Figure 7.15: Distribution of 0–100 lead scores by tier.

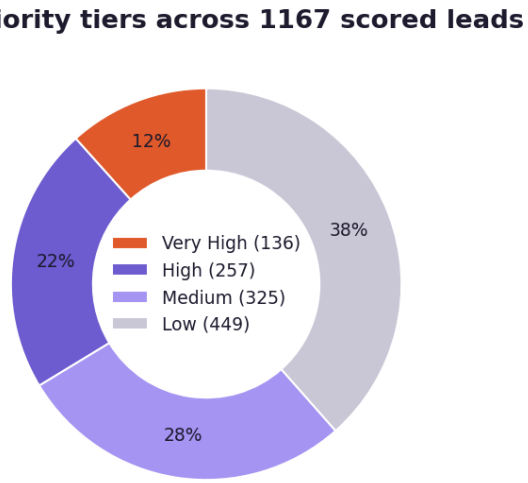


Figure 7.16: Share of leads in each priority tier.

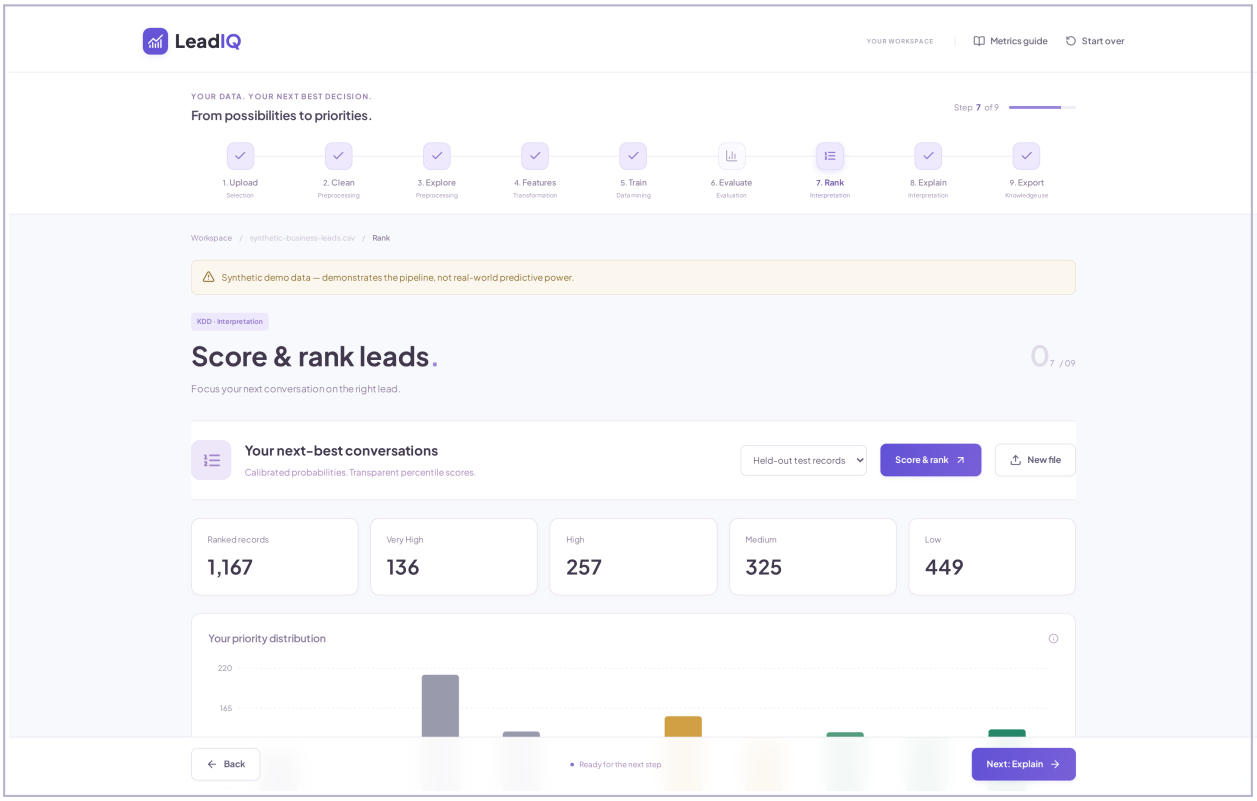


Figure 7.17: Step 7 — score cards and priority distribution.

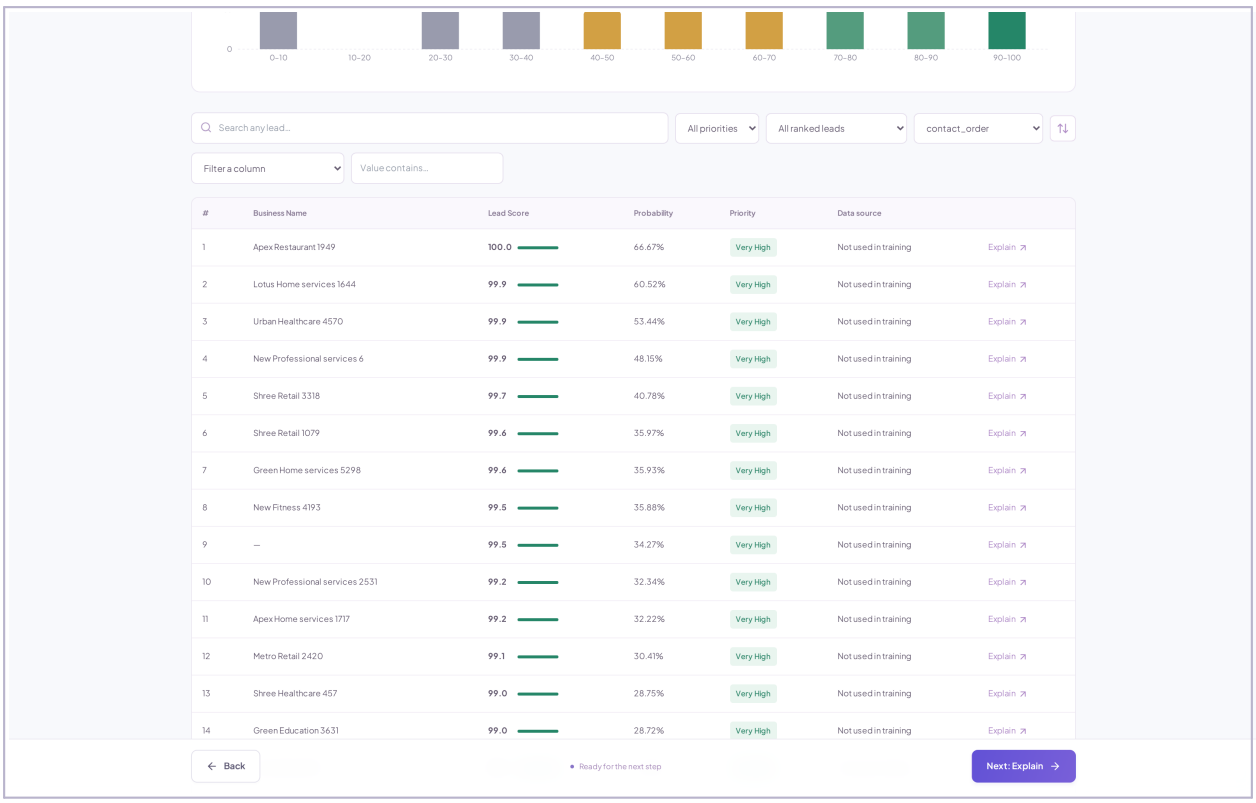


Figure 7.18: Step 7 — the ranked lead table in deterministic contact order.

7.9 Explain a Lead

For the top-ranked lead (score 100, calibrated probability 66.7%, priority Very High), SHAP shows which features pushed the prediction up or down. The interface is explicit that SHAP explains the uncalibrated estimator, not the percentile score, and that the contributions are associations rather than causation.

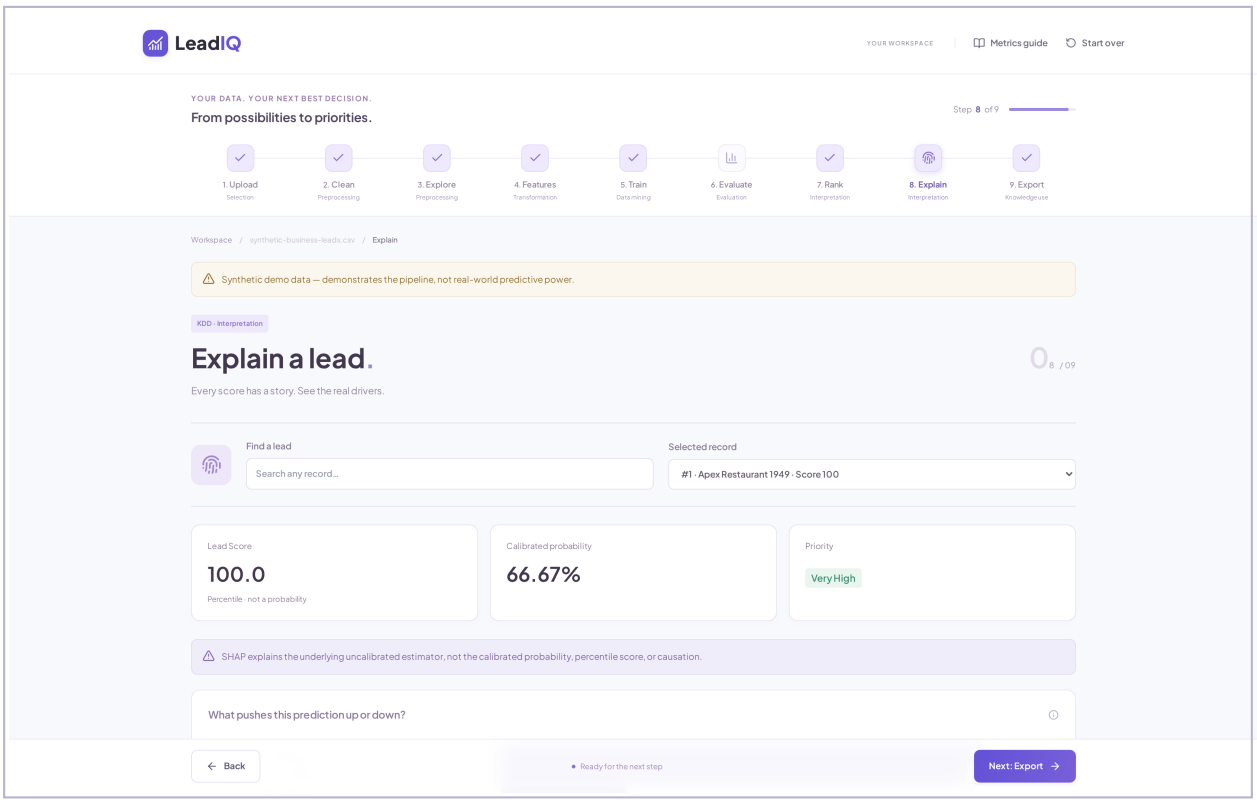


Figure 7.19: Step 8 — a per-lead SHAP explanation.

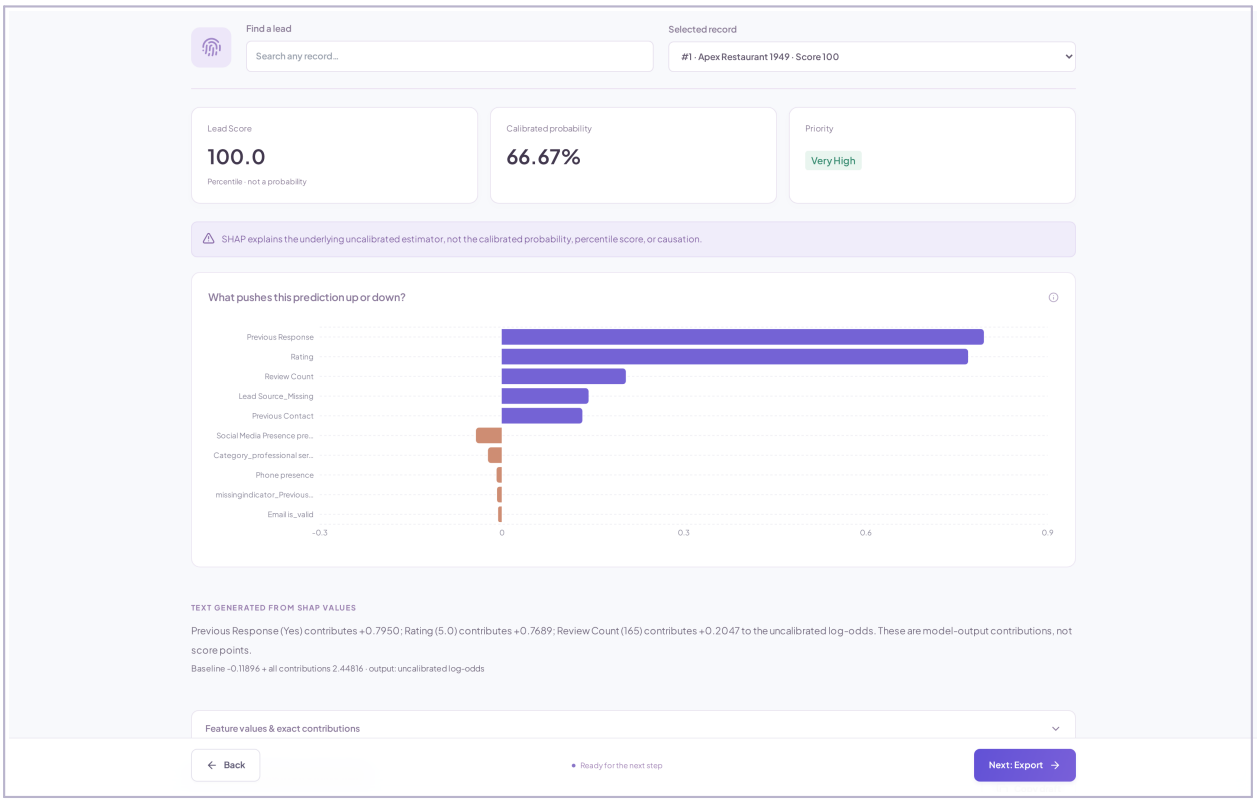


Figure 7.20: Step 8 — the SHAP contribution chart for the selected lead.

7.10 Export & Summary

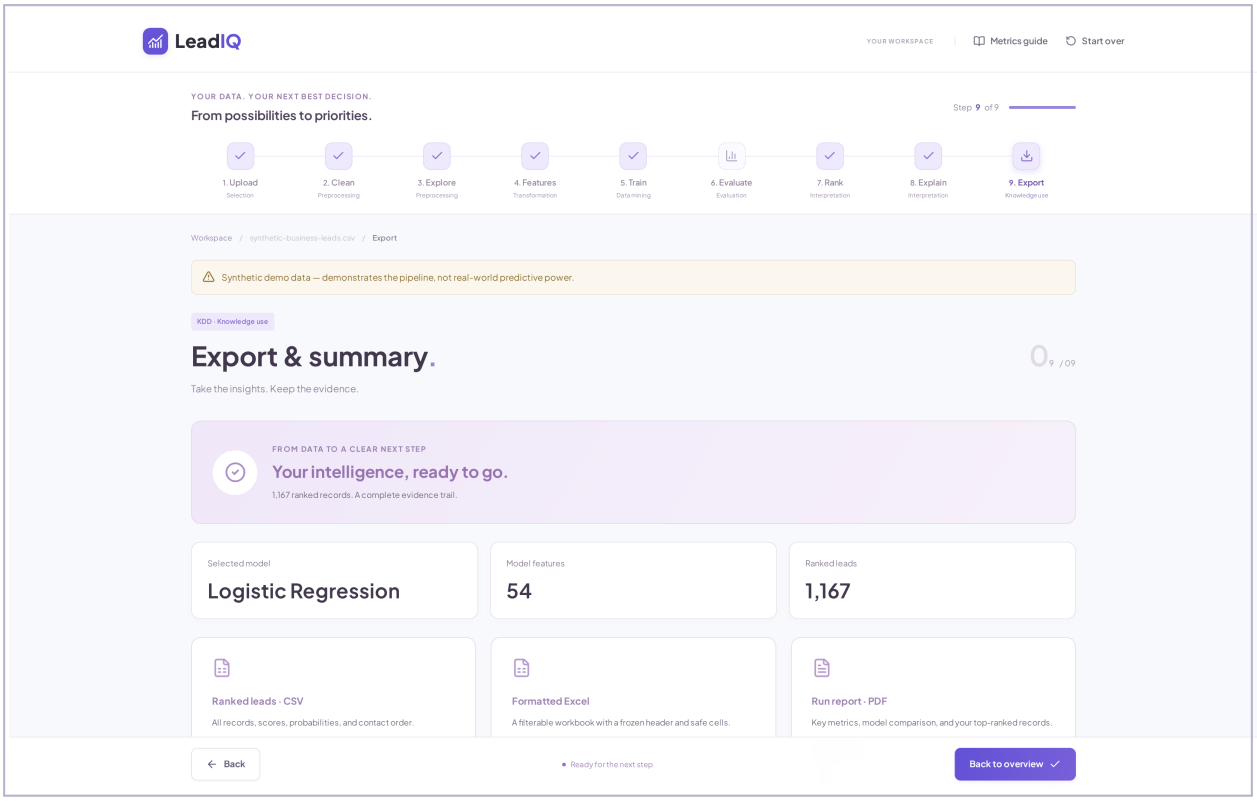


Figure 7.21: Step 9 — CSV, Excel and PDF export of the ranked leads and summary.

7.11 Discussion

The results show both the strengths and the honesty of the design. On a hard, strongly imbalanced problem the selected model reaches a PR-AUC of 0.271 against a no-information baseline of 0.099 — roughly a **17 percentage-point** improvement — and a ROC-AUC of 0.711, with about **2.95× lift** in the top decile. In practice this means a team working the top-scored leads would reach far more converters than one working the list at random. Equally important, every number is measured on data the model never saw, reported with a confidence interval, and backed by a SHAP explanation, so the user can see not only the ranking but the reasoning behind it.

8. Advantages, Limitations and Future Scope

8.1 Advantages

- **Dataset-agnostic.** Any labelled tabular file can be scored; the system is not tied to a fixed lead schema and builds its features automatically.
- **Honest evaluation.** Models are selected by training-only cross-validation and reported on an untouched test set with bootstrap confidence intervals, so the numbers are trustworthy.
- **Imbalance-aware.** Ranking metrics (PR-AUC, lift, precision in the top 10%) and a tuned operating threshold are used instead of misleading raw accuracy.
- **Calibrated probabilities.** Isotonic calibration makes a score of 0.3 really mean about a 30% chance, which matters when the output drives real decisions.
- **Explainable.** Every prediction comes with real SHAP contributions and a global importance view, with limitations disclosed rather than hidden.
- **Guided and safe.** The nine-step KDD workflow invalidates downstream results when an upstream choice changes, so stale numbers can never be mistaken for current ones.

8.2 Limitations

- The dataset used in this report is **synthetic**; it demonstrates the full pipeline but not real-world predictive power. On this hard problem the absolute precision is modest.
- Text embeddings run on CPU with the English-first *all-MiniLM-L6-v2* model; long or non-English text is truncated and may be embedded less reliably.
- SHAP and permutation importance are computed on a sample for speed, so very small differences in importance should not be over-interpreted.
- Scores reflect **association** learned from historical data, not causation, and depend on the quality and representativeness of the uploaded data.

8.3 Future Scope

- Save and compare separate historical runs side by side as an explicit history feature.
- Monitor predictions against later real-world outcomes and detect performance drift over time.
- Add GPU-accelerated and multilingual text embeddings for larger, non-English datasets.
- Support multi-class and regression targets in addition to binary classification.
- Add role-based accounts and team sharing for collaborative outreach.

9. Conclusion

LeadIQ shows that a complete, practical and **trustworthy** lead-scoring tool can be built by combining classical machine learning with careful engineering around the models. A user uploads a spreadsheet; the system cleans it, builds 54 features including sentence-transformer text embeddings, compares four classifiers against a no-skill baseline with training-only cross-validation, calibrates the winner, chooses an operating threshold, and returns a ranked, tiered contact list in which every score is explained with real SHAP values.

The main lesson of the project is the same one that runs through the whole KDD process: model output must not be trusted blindly. Reserving an untouched test set, reporting confidence intervals, preferring ranking metrics over accuracy on an imbalanced problem, calibrating probabilities, and explaining every prediction are what turn a bare classifier into a dependable decision aid. On a deliberately hard synthetic dataset the selected model still delivered about a 17 percentage-point improvement in PR-AUC over the baseline and roughly 2.95× lift in the top decile — while never hiding its uncertainty. The result is a system that answers not only “who is the next best lead?” but also “why?”.

10. References

[1] U. Fayyad, G. Piatetsky-Shapiro and P. Smyth, “From Data Mining to Knowledge Discovery in Databases,” *AI Magazine*, vol. 17, no. 3, pp. 37–54, 1996.

[2] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] scikit-learn developers, “TargetEncoder — target (mean) encoding of categorical features,” scikit-learn User Guide, https://scikit-learn.org/stable/modules/preprocessing_targetencoder.html.

[4] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.

[5] S. M. Lundberg and S.-I. Lee, “A Unified Approach to Interpreting Model Predictions,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[6] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019. (Model: all-MiniLM-L6-v2.)

[7] B. Zadrozny and C. Elkan, “Transforming Classifier Scores into Accurate Multiclass Probability Estimates,” in *Proc. 8th ACM SIGKDD*, 2002. (Isotonic calibration.)

[8] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLoS ONE*, 10(3), 2015.

[9] S. Ramírez, “FastAPI framework,” https://fastapi.tiangolo.com.

[10] Meta Open Source, “React — A JavaScript library for building user interfaces,” https://react.dev.